

Applied AI Workshop

Welcome!

Today's Agenda

Time	Activity	Lead
1:00 pm - 2:00 pm	AI Concepts	Edgar Lobaton
2:00 pm - 2:30 pm	Reflection: Exploring Code Assistants	Edgar Lobaton
2:30 pm - 3:00 pm	Presentation: Towards open autonomous labs for plant and microbial agentic phenotyping Larry York is a whole-plant and soil ecophysiologicalist at Oak Ridge National Laboratory. He develops theory, hardware, and software to increase understanding of how roots interact with microbes in soil. He has developed tools such as RhizoVision Explorer for measuring root system architectural traits and high-throughput respiration and nitrogen uptake assays. Over the past couple years, this co-development of hardware and software has led him to explore the use of robotics and large language models (LLMs) to enable autonomous phenotyping. This talk will focus on open-source robotic phenotyping and new data analysis benchmarks using local LLMs.	Larry York, ORNL
3:00 pm - 3:15 pm	Coffee Break	
3:15 pm - 3:45 pm	Tool Overview: Virtualization	Edgar Lobaton
3:45 pm - 4:15 pm	Presentation: Pedagogical Perspective	Edgar Lobaton
4:15 pm - 4:45 pm	Hands on: GitHub	Tarek Aziz
4:45 pm - 5:00 pm	Open Discussion Survey	Edgar Lobaton

From Tensors to Agents

Edgar Lobaton, Ph.D.

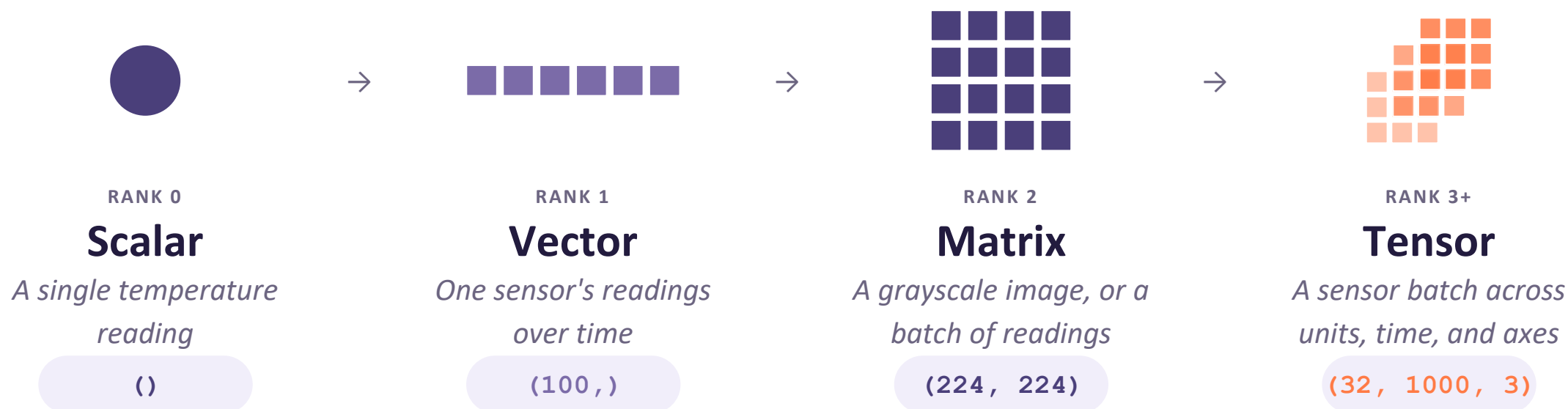
Director of Applied AI, College of Engineering (COE), NC State
Professor, Electrical and Computer (ECE) Engineering, NC State
NAIRR AI Education Fellow, Computing Research Association (CRA)

Roadmap

- We open the black box behind
predictive (insight) → generative (content) → agentic (action)
- Predictive and Generative workflows
Text / Images → Tensors → Models → Prediction
Text → Tensors → Models → Text / Image Generation
- Getting to an Agentic Pipeline
Generation → Reasoning → Actions

What is a Tensor?

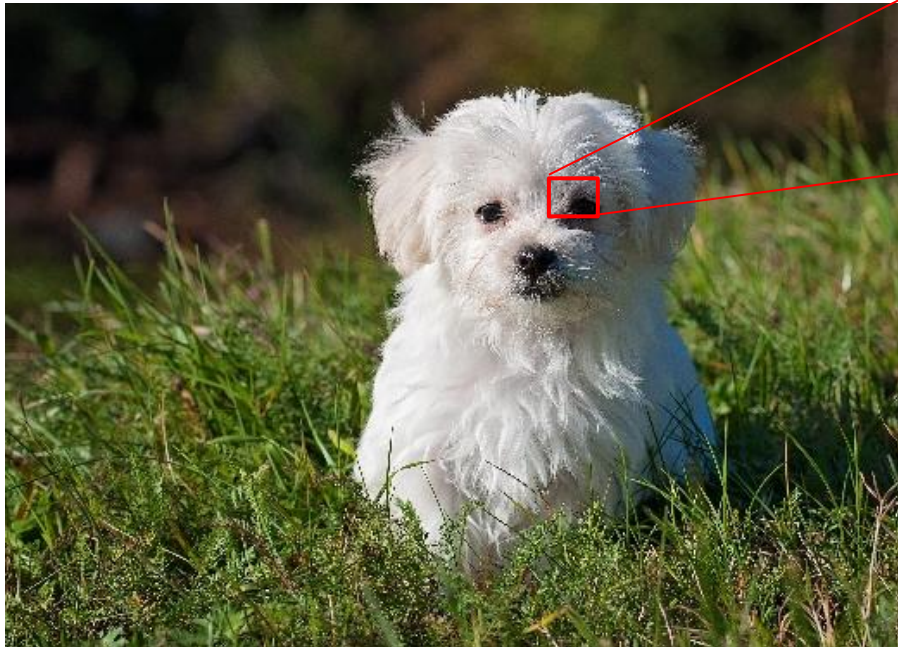
- A multi-dimensional (n-dimensional) array of numbers. Think of the generalization of a vector and a matrix.



- Every step in an AI/ML workflow uses tensor operations under the hood.

From Images to Tensors

Images are arrays of pixels value. All that the computer “sees” is a list of those numbers.



Shape = (8,12,3)

Predictions could be class predictions:

$Cat \rightarrow [1 \ 0 \ 0]$; $Dog \rightarrow [0 \ 1 \ 0]$; $Bird \rightarrow [0 \ 0 \ 1]$

Object detections:

$Dog \rightarrow [x_0, y_0, x_1, y_1, c]$

Image Segmentation

Content Generation

From Text to Tensors



Tokenization

Text is broken into subword units (tokens), each mapped to an integer ID — not quite words, not quite characters.

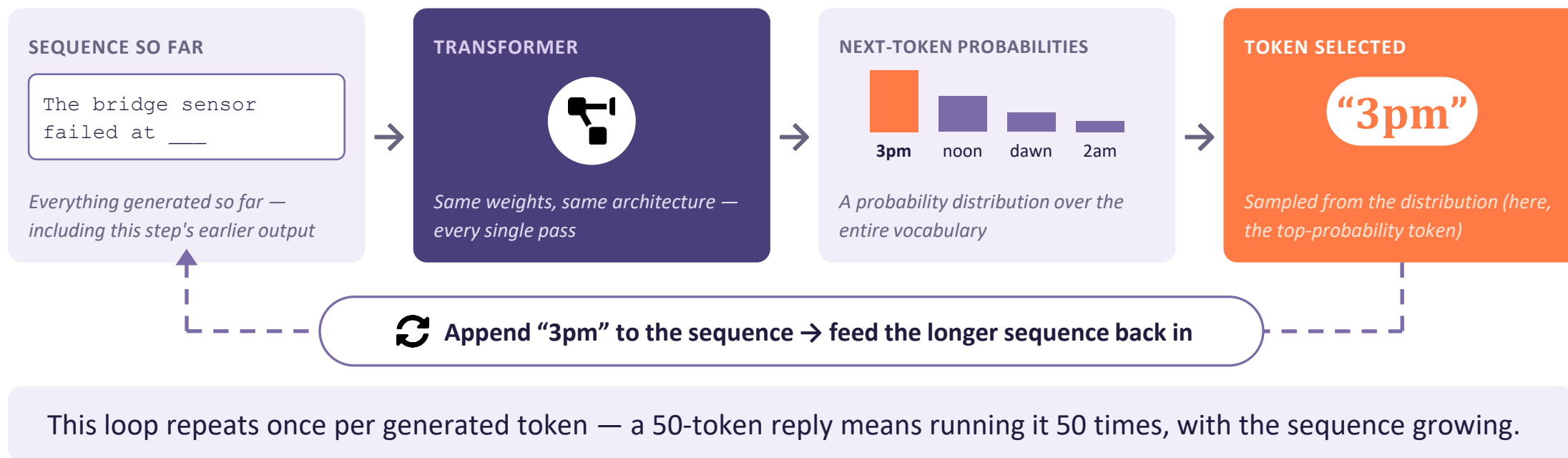
Embeddings

Each token ID is mapped to a vector in high-dimensional space, positioned so related meanings land near each other.

- Sequences of vectors are then used as input to models that can predict class labels, sequence labeling, etc.

Generating Text

- Each new token is predicted, appended and fed back in.



- The transformer itself is stateless – all memory lives in the growing sequence fed back in real time. The **context-window** is the input to the transformer. It has to be bounded.

From Images to Tensors (Revisited)

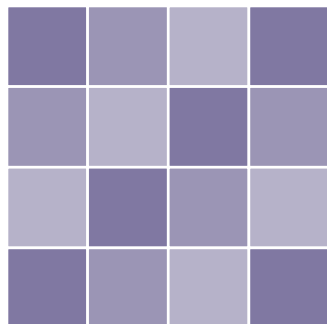
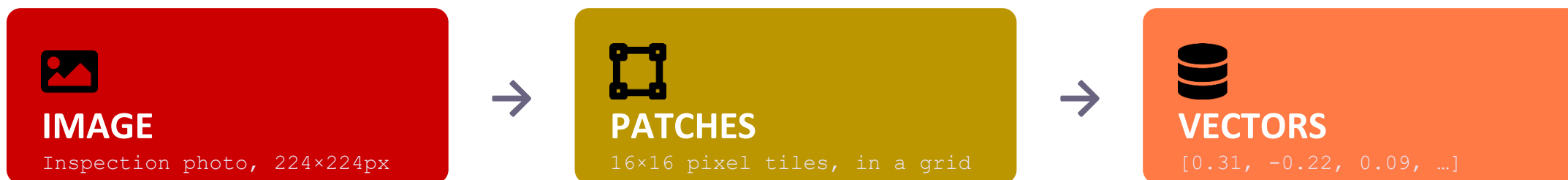


Image divided into a grid of patches

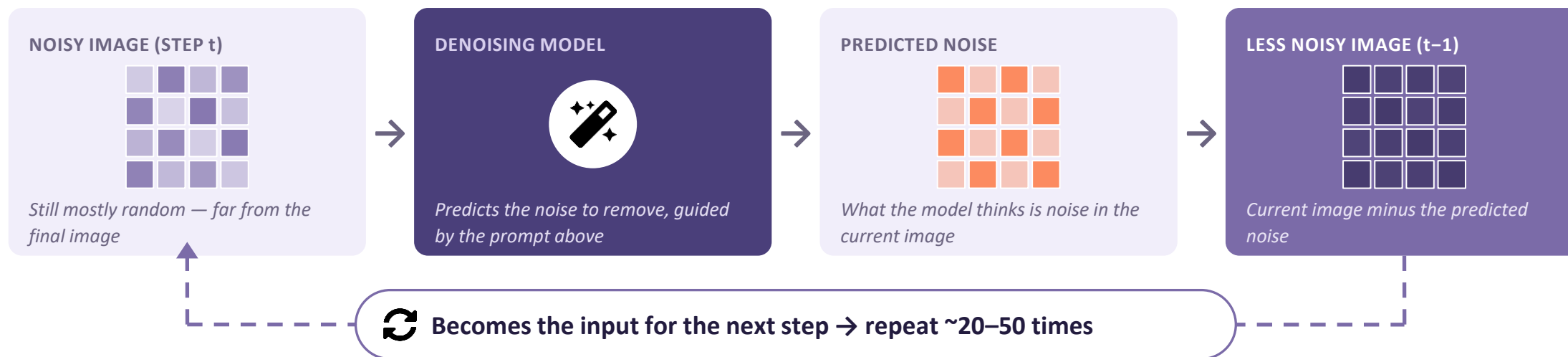
Each patch (or, in older CNN approaches, each learned filter response) becomes a vector — the same embedding-space idea from text, just applied to pixels instead of tokens.

- Input images don't need to be processed sequentially over space.

Generating Images

- Each pass removes a little noise and feeds the results back in

A TEXT PROMPT: “a reinforced concrete bridge at sunset” — the same prompt conditions every single step below



This loop repeats once per denoising step — a typical image takes 20–50 steps.

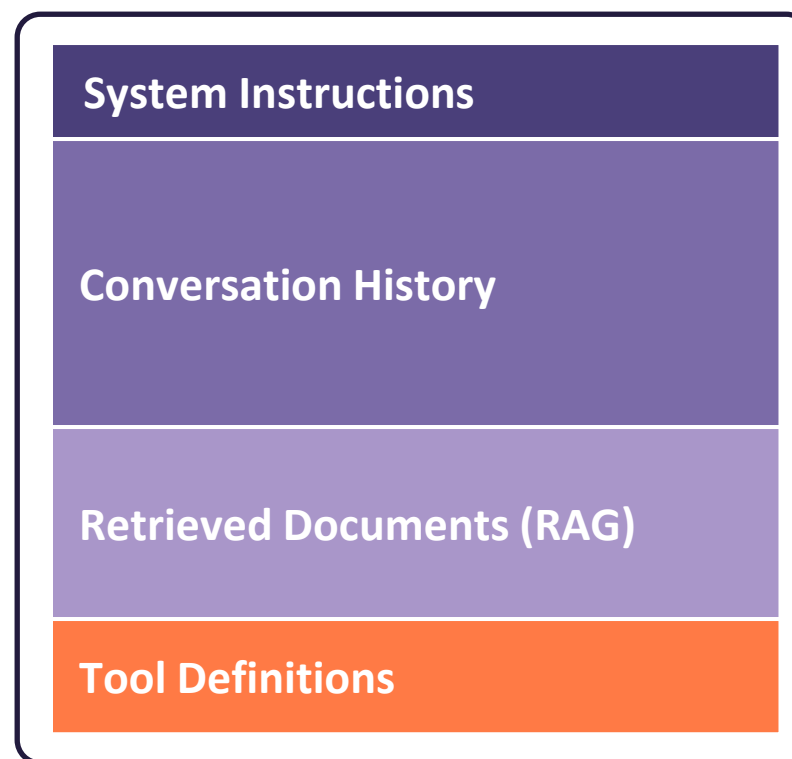
- The denoising model is stateless – all memory lives in the image tensor passed between steps.

From Generative Model to Assistant

Context: The Model's Working Memory

- Everything the model “knows” right now lives in this one limited space
- It has no persistent memory between calls unless something explicitly puts information back into context
- Context Engineering decides what goes into this limited space, and in what order
- Once something falls outside the window, the model has no mechanism to say: “I don’t have that”. It just continues generating plausible content.

CONTEXT WINDOW



Personas: Shaping Behavior, Not Knowledge

- The same question, the same model, two very different answers



“Patient Tutor”

Q: Why isn't my gradient descent converging?

Great question — let's work through it. First, check your learning rate: if it's too high, updates can overshoot the minimum and bounce around instead of settling. Try reducing it by 10x and see what happens...



“Terse Code Reviewer”

Q: Why isn't my gradient descent converging?

Learning rate too high. Reduce by 10x. Also check for unnormalized features — likely culprit given your loss curve.

Hallucinations: Confident, Not Necessarily Correct

- The model predicts plausible text. It has no built-in “I don’t know” signal.



Model's apparent confidence

Consistently high — every answer sounds equally certain

Actual factual accuracy

Varies — depends on training data and topic

- The model is producing the statistically plausible next token, not consulting a fact database. When it doesn’t “know” something, it just continues the pattern rather than stopping.

RAG vs. Fine-Tuning

- RAG (Retrieval-Augmented Generation) is really an automated way of deciding what goes into Context — based on relevance, not fixed in advance.

	RAG	Fine-Tuning
What changes	Context, at request time	The model's weights
Update speed	Instant — swap the source	Requires retraining
Source attribution	Yes — traceable to a retrieved doc	No — baked into weights
Best for	Fast-changing or large knowledge bases	Stable shifts in behavior or style

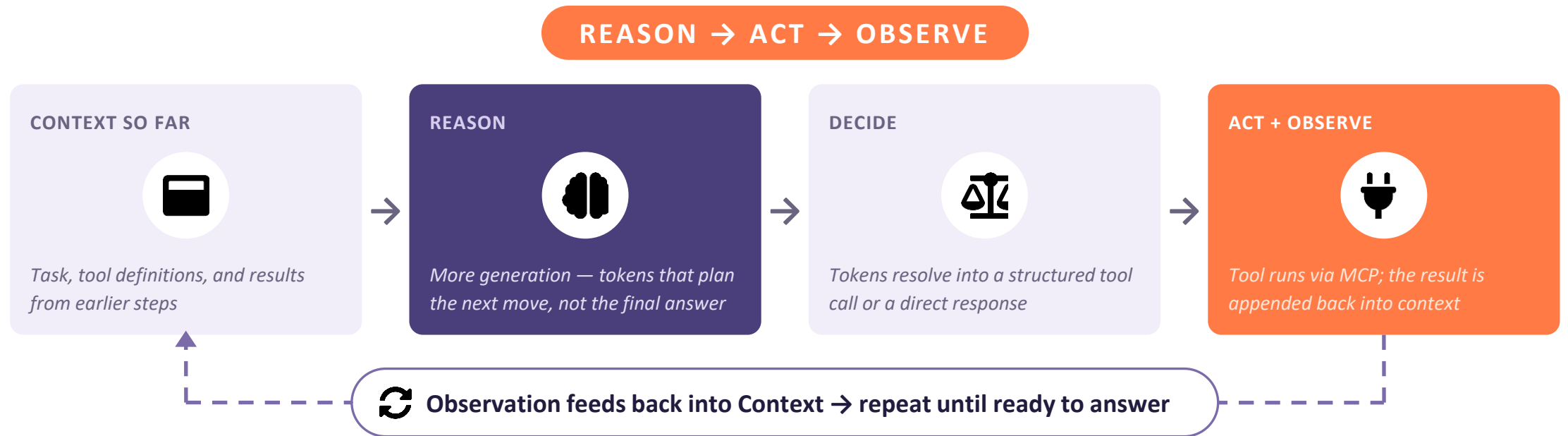
From Generative Models to Assistants

<https://forms.gle/QBSBH5r56fKstaSy7>



From Generation to Action

Reasoning

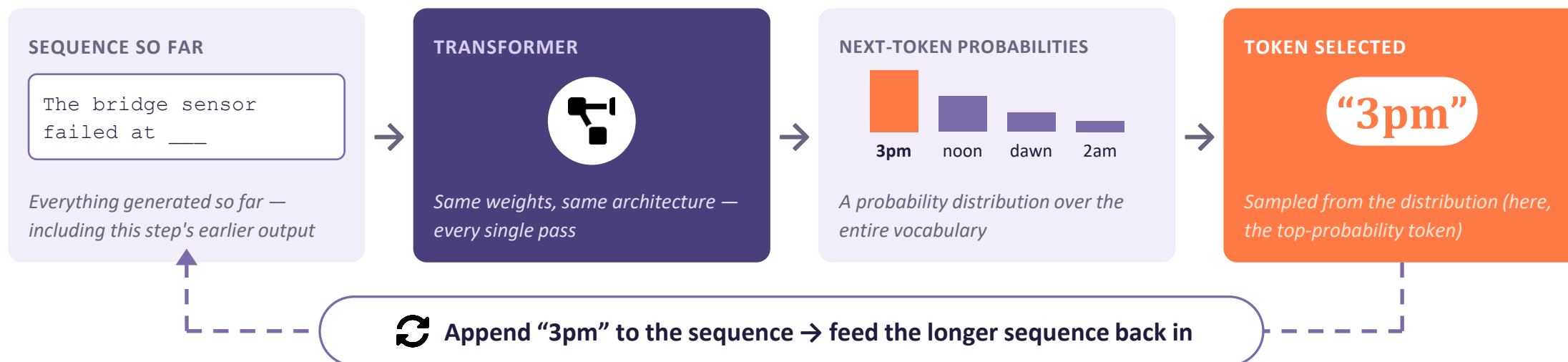


This loop can run many times — each pass may call a different tool, refine the plan, or decide it's ready to answer.

- Does this look familiar?

Generating Text

- Each new token is predicted, appended and fed back in.



This loop repeats once per generated token — a 50-token reply means running it 50 times, with the sequence growing.

- The transformer itself is stateless — all memory lives in the growing sequence fed back in real time. The **context-window** is the input to the transformer. It has to be bounded.

Skills: Packaged Procedural Expertise

- Reusable knowledge the model reads before doing a specific class of tasks



Instructions, examples, templates — read before the task, not baked into weights

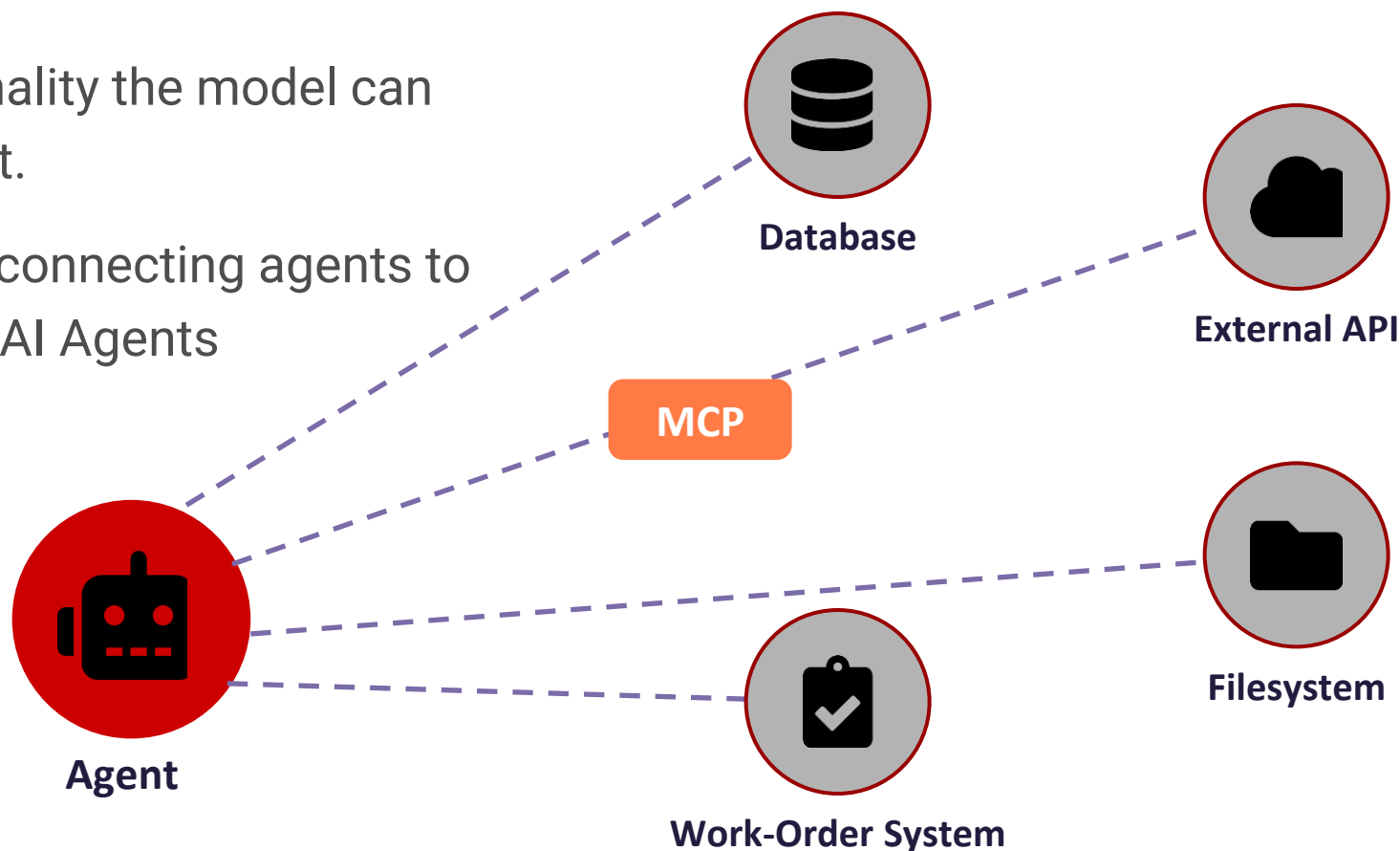
Engineering analogy: a skill is functionally a standard operating procedure (SOP). A specialist doesn't re-derive the correct inspection procedure from first principles each time — they follow a written procedure refined over many iterations.

This is now a first-class concept in production agent platforms — not just an informal prompting trick.

- While Personas set the role, Skills supply the know-how for a specific tasks within that role

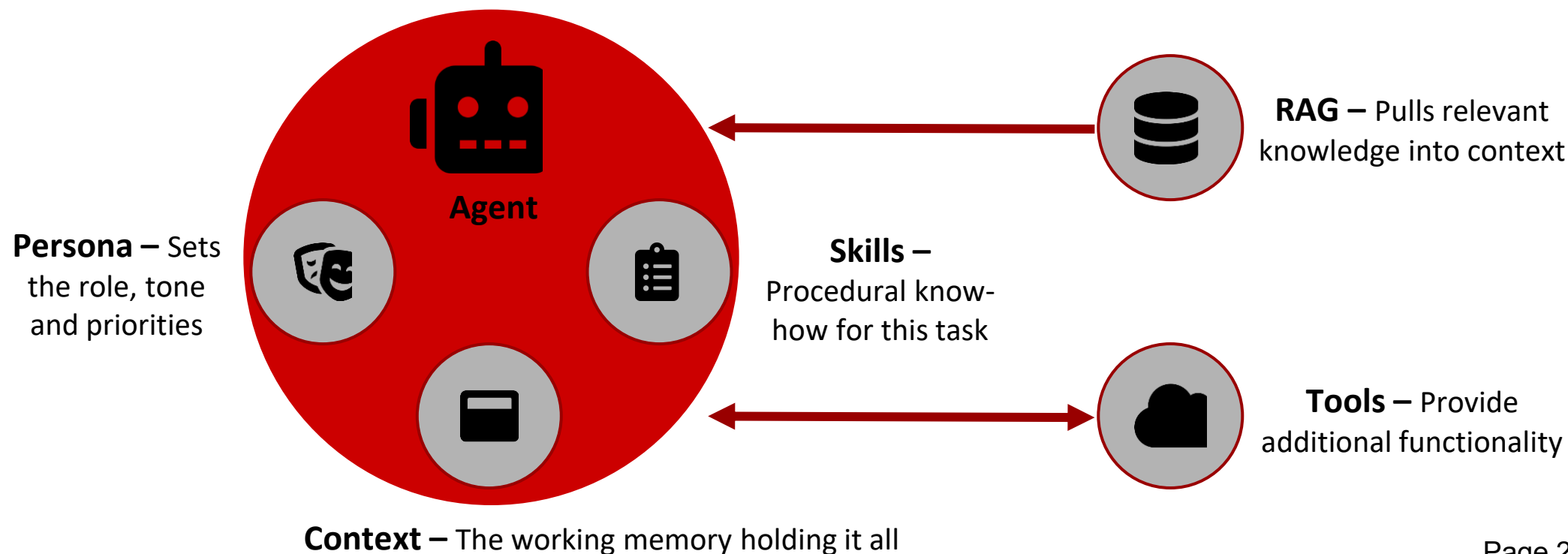
MCP: Model Context Protocol

- A tool is any external functionality the model can invoke beyond generating text.
- MCP is an open standard for connecting agents to tools and data, “the USB-C of AI Agents”



Putting It All Together

- An agent incorporates a persona, with skills, calling tools via MCP, grounded by RAG, making repeated generative calls in a loop.



From Generation to Action

<https://forms.gle/3urW27unTHHuF3yH8>



Let's Recap